

Fetching Google News with an API, and Landing It in Azure

A step-by-step walkthrough of pulling live Google News headlines through SearchAPI.io and automatically saving them into Azure Data Lake Storage using Azure Data Factory.

What this project does

In short: I use a free API called **SearchAPI.io** to fetch the latest Google News headlines, then I build a pipeline in **Azure Data Factory** that calls that API automatically and saves the result as a JSON file in **Azure Data Lake Storage**. No manual downloading, no copy-pasting — once it's set up, the pipeline does the fetching for you.

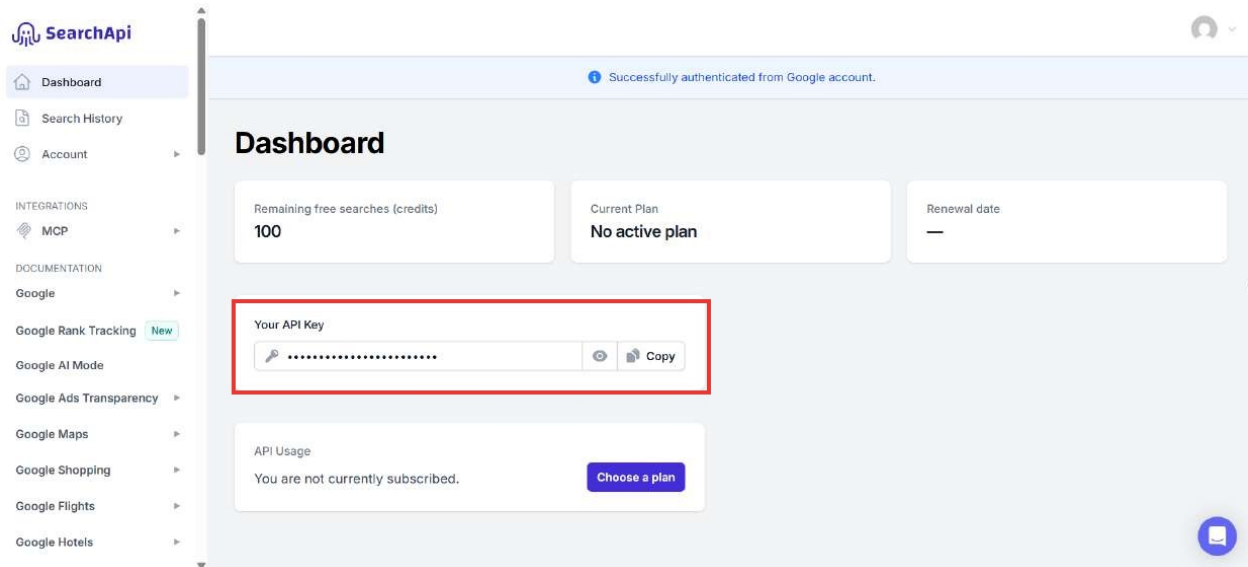
It's split into four parts:

1. Get a working API URL from SearchAPI.io (that fetches Google News for India)
2. Set up a storage account in Azure to hold the results
3. Build the Data Factory pipeline that connects the two
4. Run it and check that the news actually landed in storage

Part 1 — Building the Google News API URL

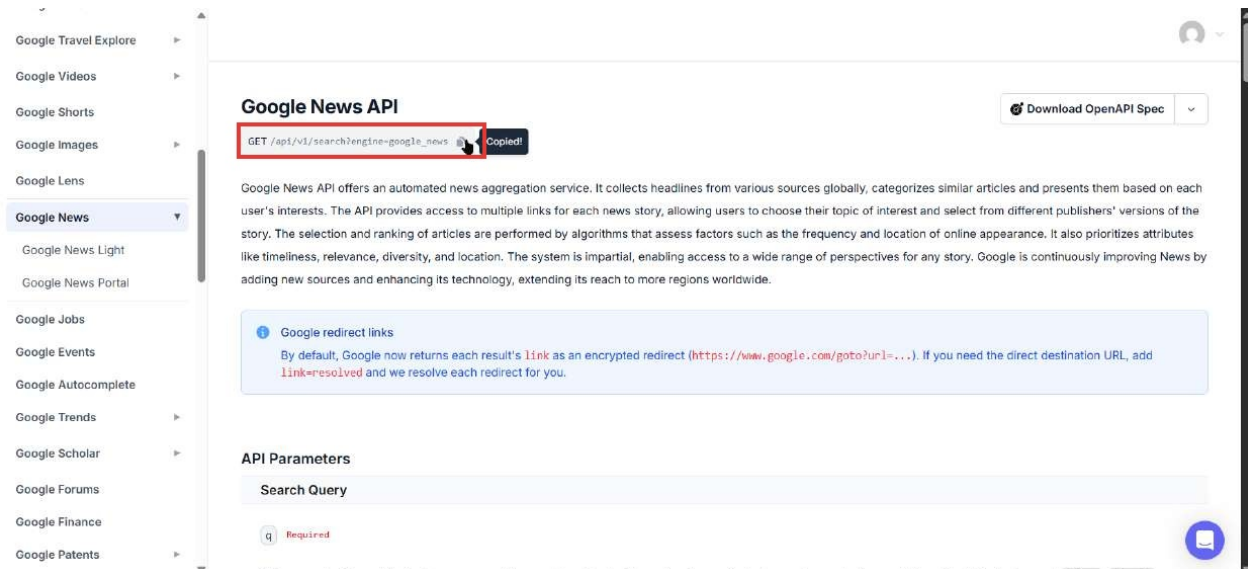
SearchAPI.io is a service that turns Google searches (News, Maps, Shopping, etc.) into a simple API call. Before touching Azure at all, the first job is to figure out exactly what URL to call so it returns Google News results for India.

Step 1: Sign up and grab your API key



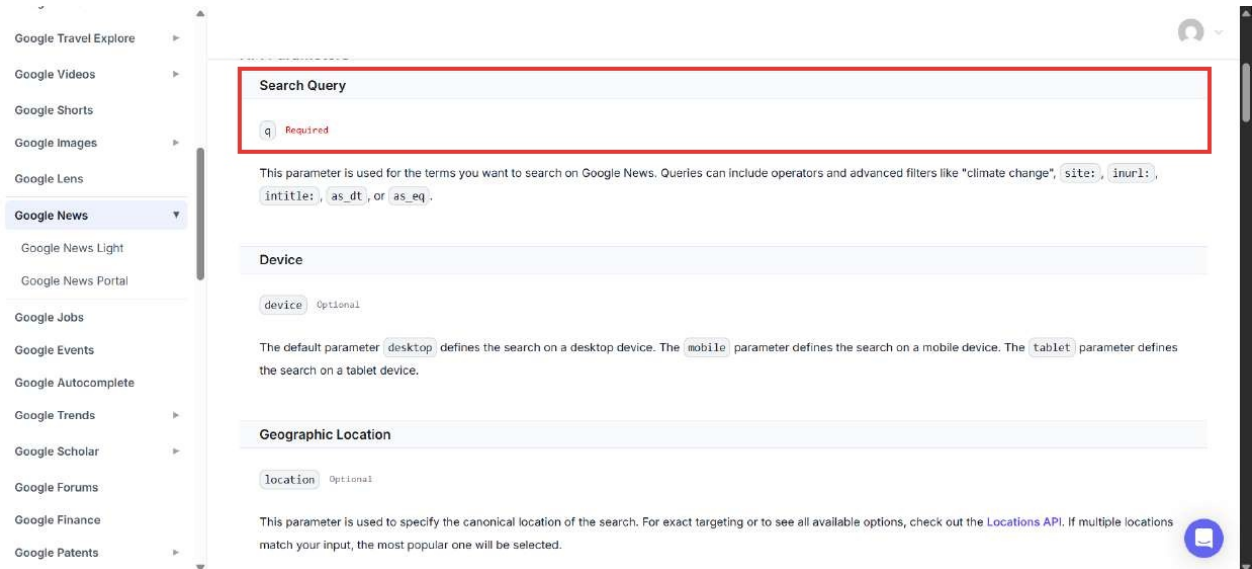
Every account comes with a free tier (100 searches to start) and a personal API key on the Dashboard. That key gets added to every request you make — it's how SearchAPI knows the request is yours.

Step 2: Find the Google News endpoint



Each Google product SearchAPI supports has its own documentation page. The Google News page gives you the base endpoint to copy: `GET /api/v1/search?engine=google_news`. The `engine=google_news` part is what tells SearchAPI which Google product to search.

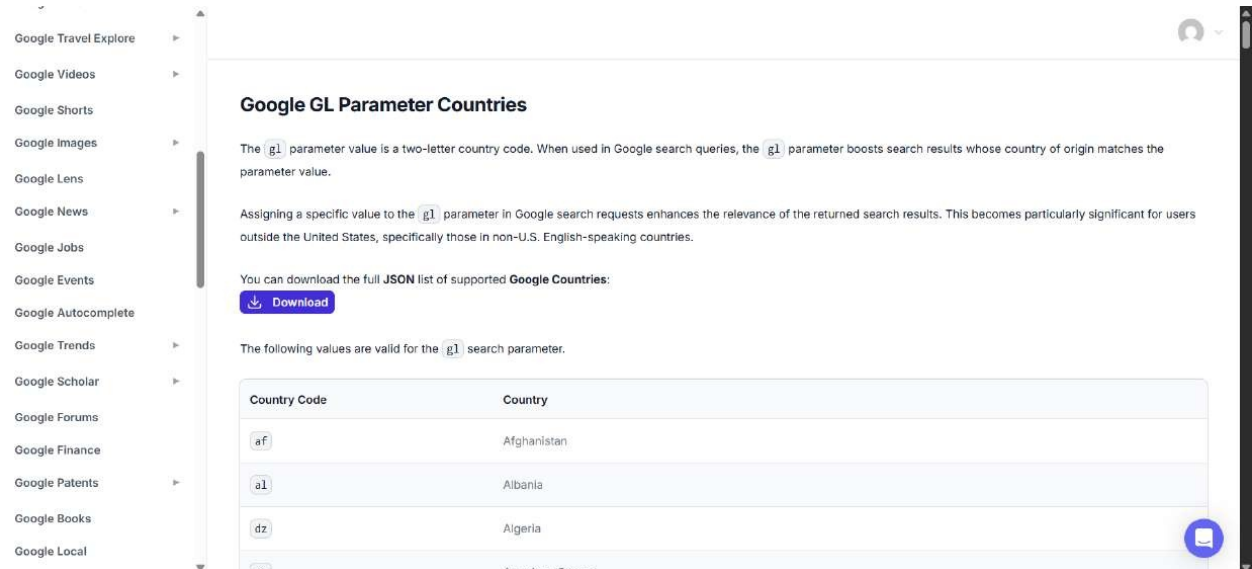
Step 3: Check the required parameter



The screenshot shows the Google News API documentation page. On the left is a sidebar with a list of Google services: Google Travel Explore, Google Videos, Google Shorts, Google Images, Google Lens, Google News (highlighted), Google News Light, Google News Portal, Google Jobs, Google Events, Google Autocomplete, Google Trends, Google Scholar, Google Forums, Google Finance, and Google Patents. The main content area is titled 'Search Query' and is highlighted with a red rectangular box. Below this title, the parameter 'q' is listed as 'Required'. A text block explains that this parameter is used for the terms you want to search on Google News, and provides examples of operators and filters like 'climate change', 'site:', 'inurl:', 'intitle:', 'as_dt', or 'as_eq'. Below the 'Search Query' section, there are sections for 'Device' and 'Geographic Location', each with an optional parameter ('device' and 'location' respectively) and a brief explanation of their use.

Scrolling down the same page shows the parameters you can use. Only one is actually required: `q` — the search term. For this project I used `q=latest+news` to pull general headlines rather than a specific topic.

Step 4: Understand the `gl` (country) parameter

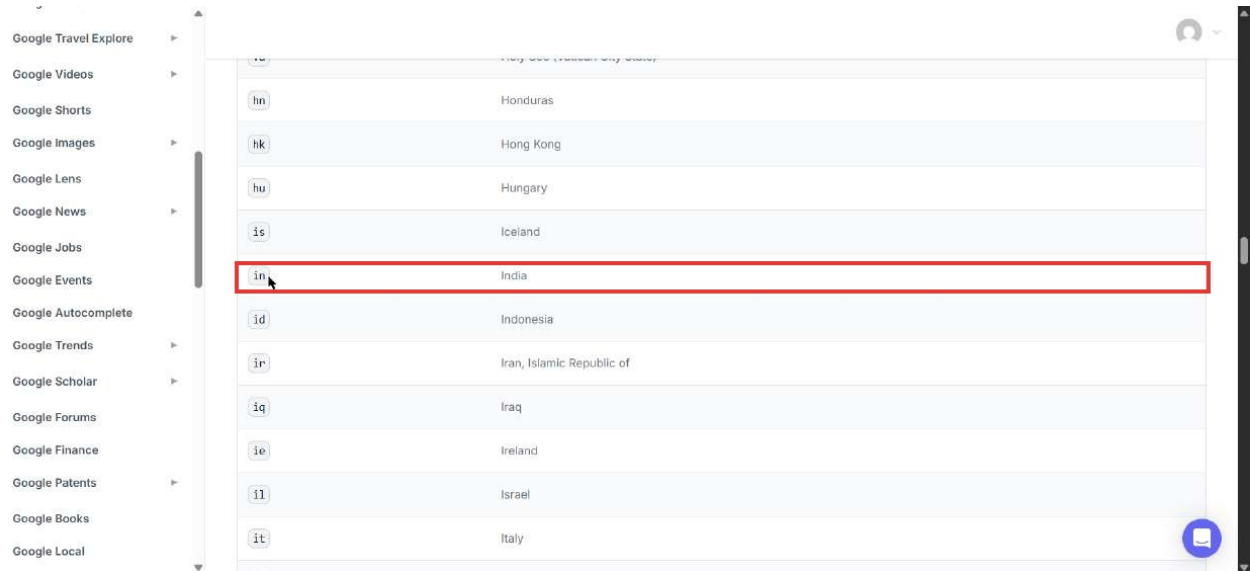


The screenshot shows the 'Google GL Parameter Countries' page. The sidebar on the left is the same as in the previous screenshot, with 'Google News' highlighted. The main content area is titled 'Google GL Parameter Countries'. It explains that the `gl` parameter value is a two-letter country code and that it boosts search results whose country of origin matches the parameter value. It also states that assigning a specific value to the `gl` parameter enhances the relevance of the returned search results, particularly for users outside the United States. Below this text, there is a link to download the full JSON list of supported Google Countries. A table follows, listing valid values for the `gl` search parameter. The table has two columns: 'Country Code' and 'Country'.

Country Code	Country
af	Afghanistan
al	Albania
dz	Algeria
ac	American Samoa

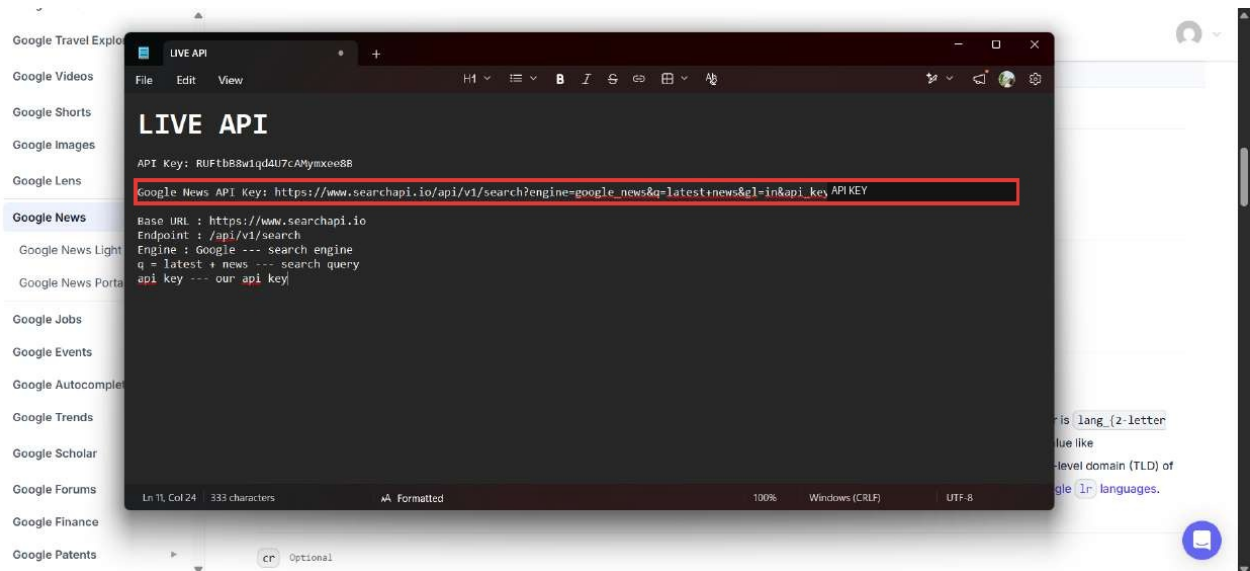
`gl` is an optional parameter that biases the results toward a specific country — handy since "latest news" means something different in every country. SearchAPI's docs list every supported country code.

Step 5: Pick India's country code



Scrolling that country list down to India shows the code is simply `in`. So the parameter becomes `gl=in` — this makes sure the headlines returned are relevant to an Indian audience.

Step 6: Put it all together



Before: just a bunch of separate pieces — a base URL, an engine name, a query, a country code, and an API key.

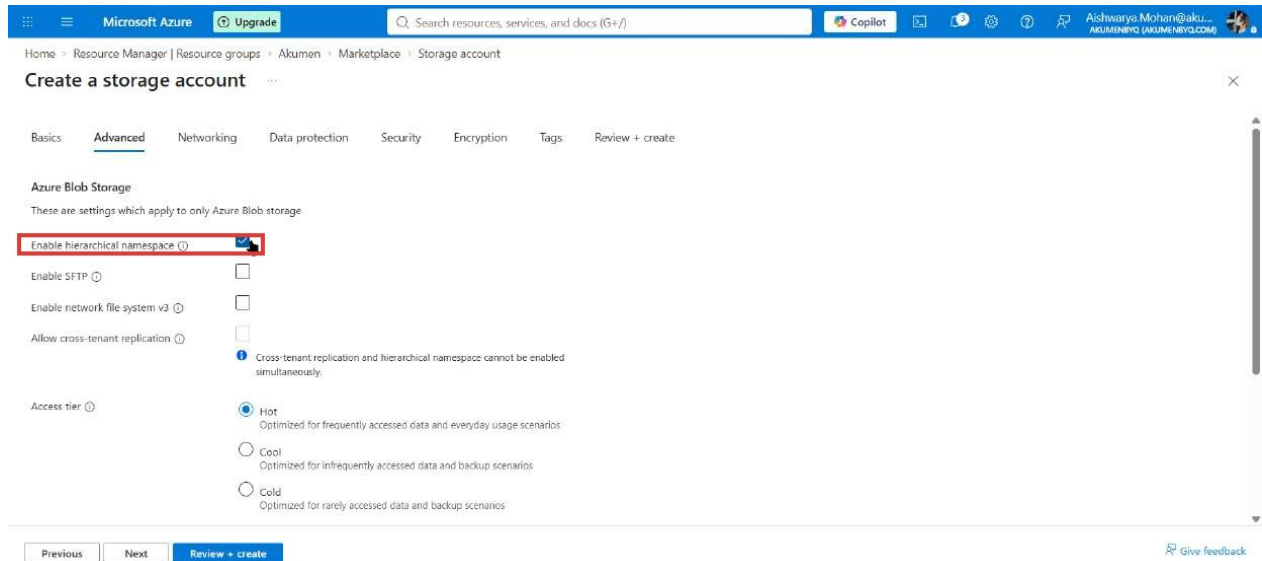
After: one working URL that returns Google News (India) as JSON the moment you open it in a browser: `https://www.searchapi.io/api/v1/search?engine=google_news&q=latest+news&gl=in&api_key=<your key>`

(The key is blurred above since this project lives on a public GitHub repo — never commit a real API key to a public place!)

Part 2 — Setting Up Storage in Azure

Now that there's a working API URL, the next question is: where should the results land? That's a storage account in Azure, with a special setting turned on so it behaves as a proper Data Lake.

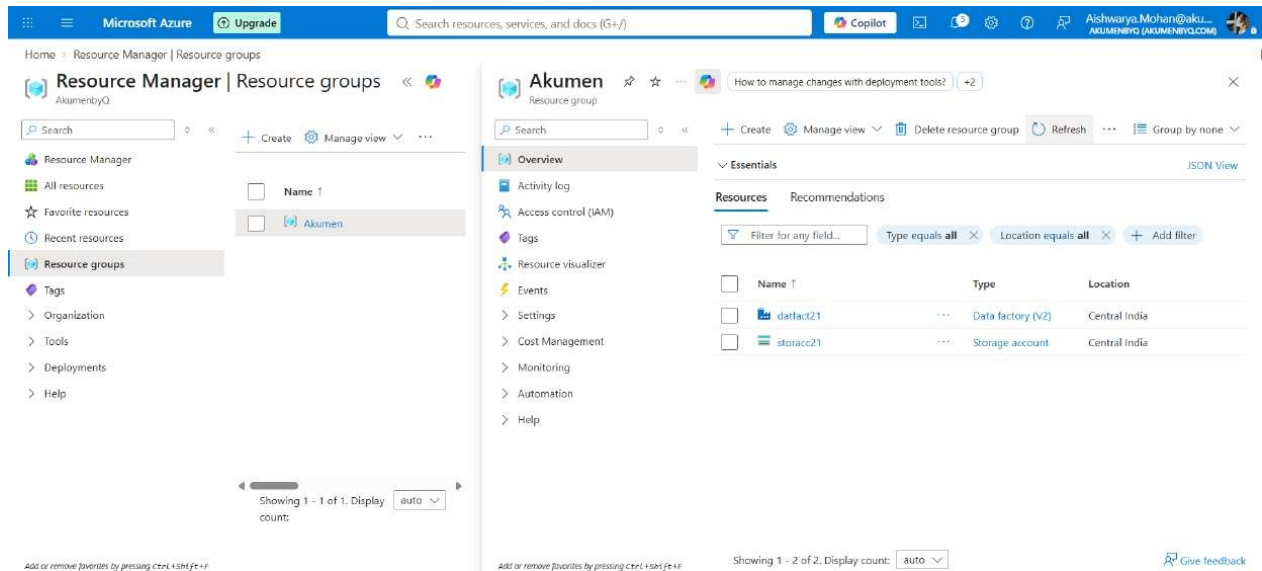
Step 7: Turn on hierarchical namespace



The screenshot shows the 'Create a storage account' page in the Azure portal, specifically the 'Advanced' tab. The page is titled 'Create a storage account' and has a breadcrumb trail: Home > Resource Manager > Resource groups > Akumen > Marketplace > Storage account. The 'Advanced' tab is selected, showing settings for 'Azure Blob Storage'. A red box highlights the 'Enable hierarchical namespace' checkbox, which is checked. Below it are three other checkboxes: 'Enable SFTP', 'Enable network file system v3', and 'Allow cross-tenant replication', all of which are unchecked. A message states: 'Cross-tenant replication and hierarchical namespace cannot be enabled simultaneously.' Under the 'Access tier' section, the 'Hot' option is selected, with descriptions for 'Hot', 'Cool', and 'Cold' tiers. At the bottom, there are 'Previous', 'Next', and 'Review + create' buttons. A 'Give feedback' link is also present in the bottom right corner.

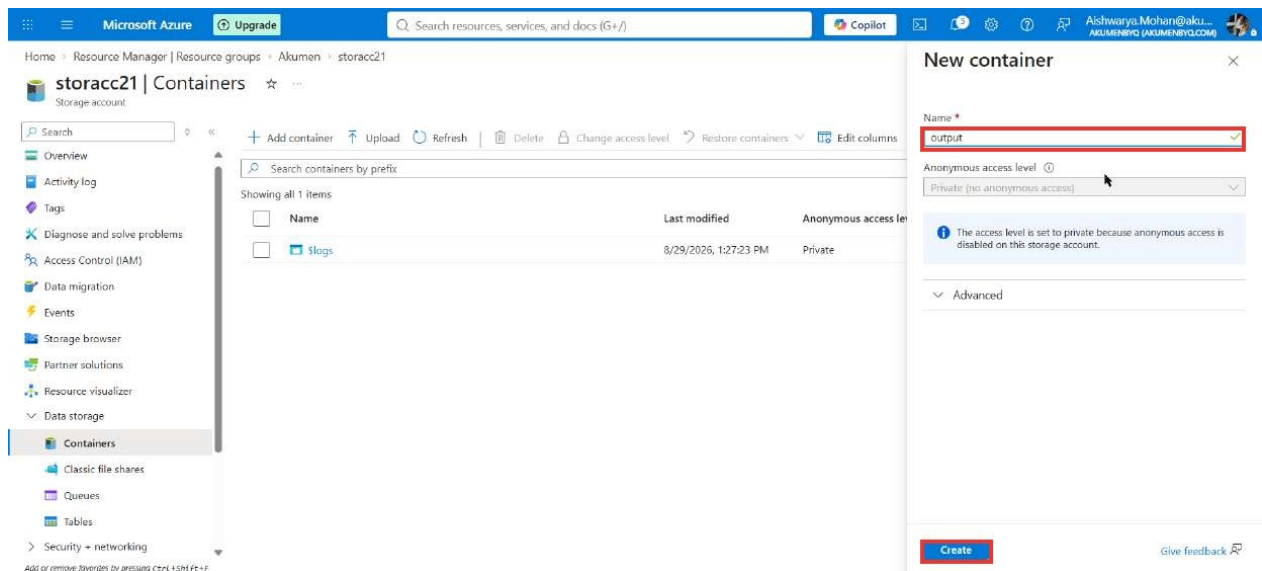
When creating the storage account, this one checkbox under the Advanced tab is what upgrades a plain Blob Storage account into **Azure Data Lake Storage Gen2**. It lets the storage behave like a folder system (directories inside directories) instead of one flat bucket — which matters later when we auto-organize files by date.

Step 8: Check the resource group



Everything for this project lives inside one resource group called **Akumen**: the Data Factory (datfact21) and the storage account (storacc21) sit side by side, so they're easy to find and clean up together later.

Step 9: Create a container named "output"

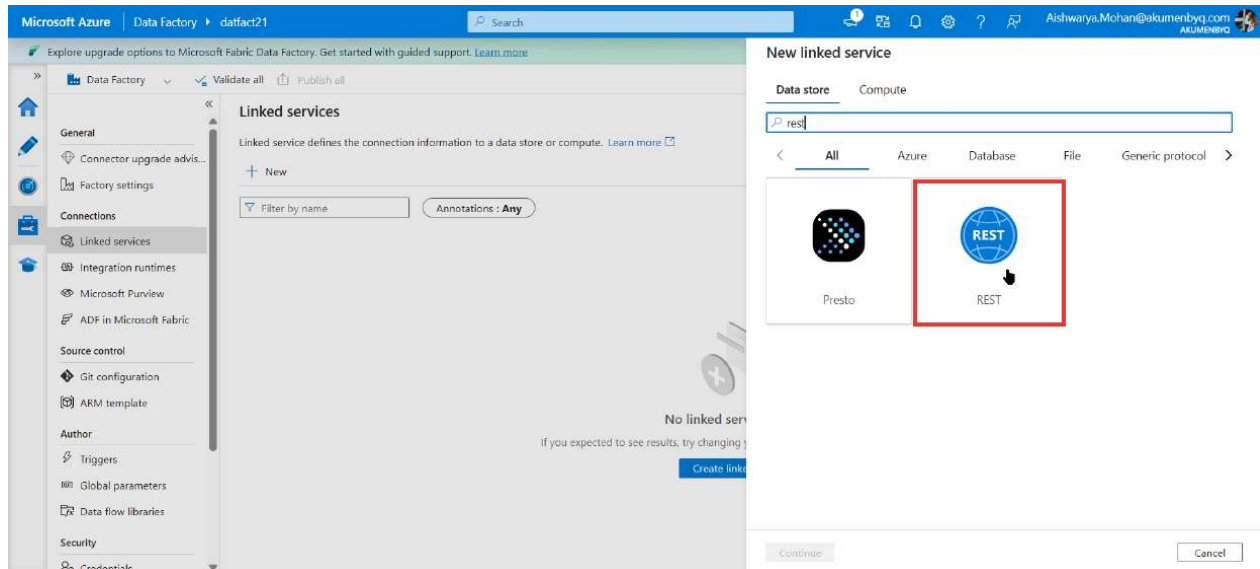


A container in a storage account is like the top-level folder. I named mine output since that's exactly what it'll hold — the output of the pipeline. Access level is left as Private, since there's no reason for this data to be public.

Part 3a — Connecting Data Factory to Both Sides

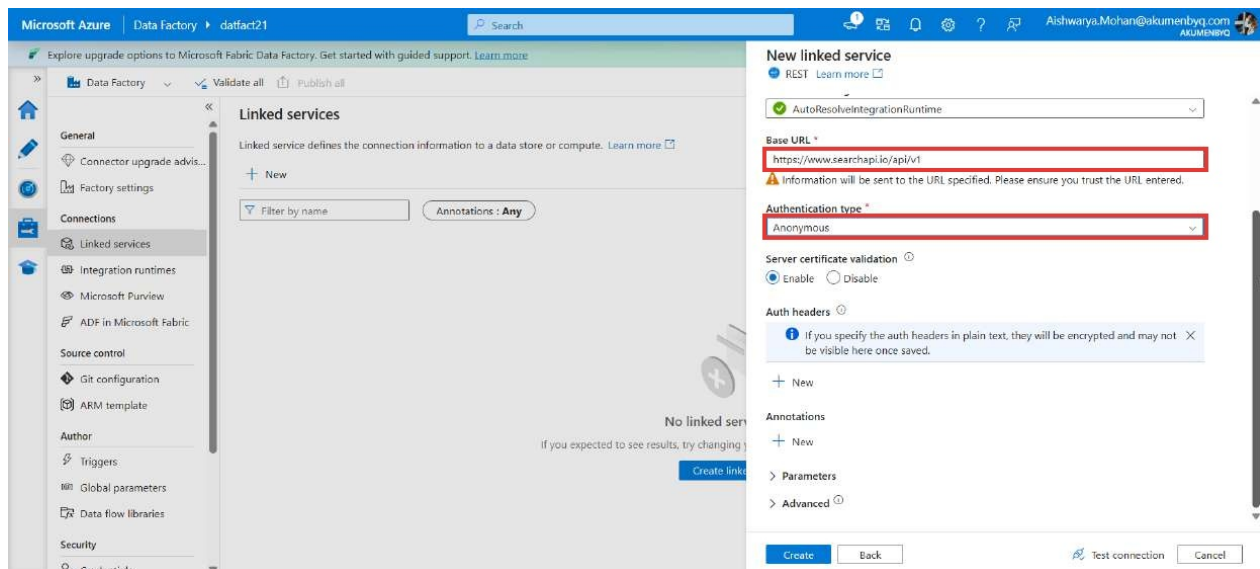
In Azure Data Factory, before you can move data from A to B, you need to tell it how to reach both A and B. These connections are called **Linked Services**. We need one for the API (the source) and one for the storage account (the destination).

Step 10: Add a REST linked service (for the API)



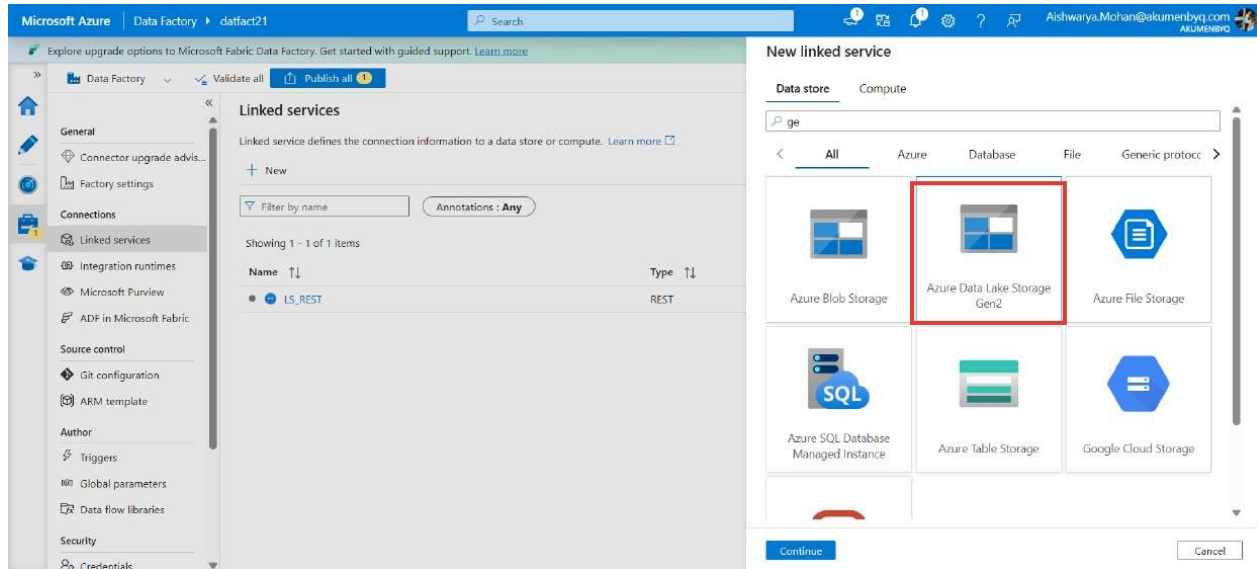
Under Linked Services, clicking New and searching "rest" brings up the REST connector — this is the generic connector for calling any API over HTTP, which is exactly what SearchAPI is.

Step 11: Point it at SearchAPI



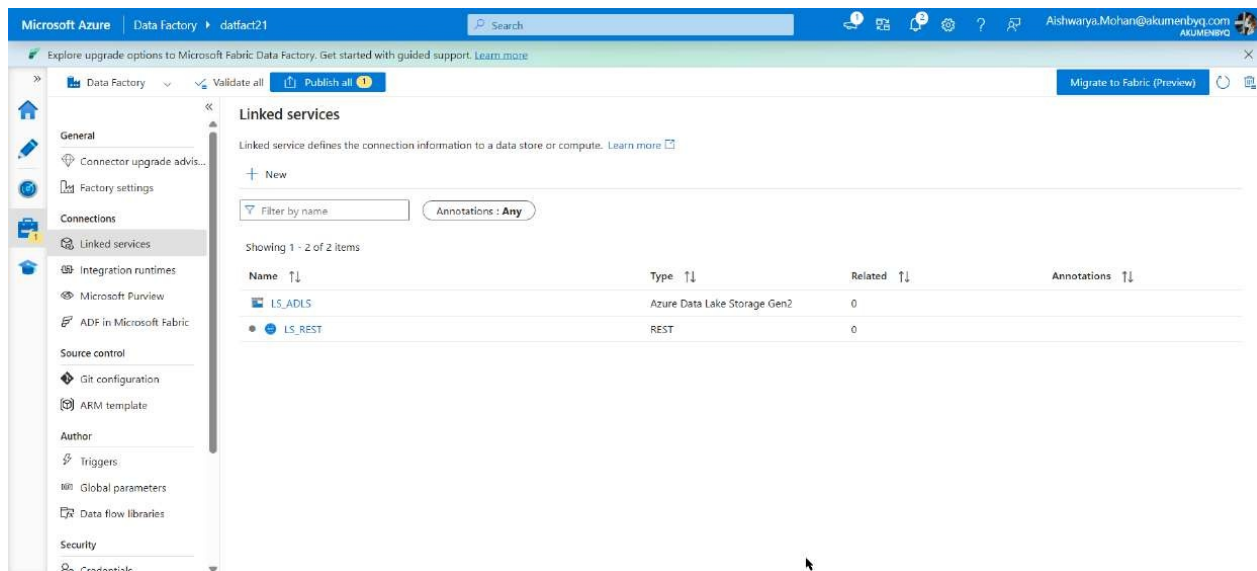
Base URL is set to `https://www.searchapi.io/api/v1` — just the fixed starting part of the URL. **Authentication type** is left as Anonymous, because instead of sending the key as a login/header, it gets tacked onto the end of the actual request URL later (in the dataset, not here).

Step 12: Add an ADLS Gen2 linked service (for storage)



The second connection points at the storage account itself, using the **Azure Data Lake Storage Gen2** connector — the same one that got unlocked by enabling hierarchical namespace back in Step 7.

Step 13: Both connections ready



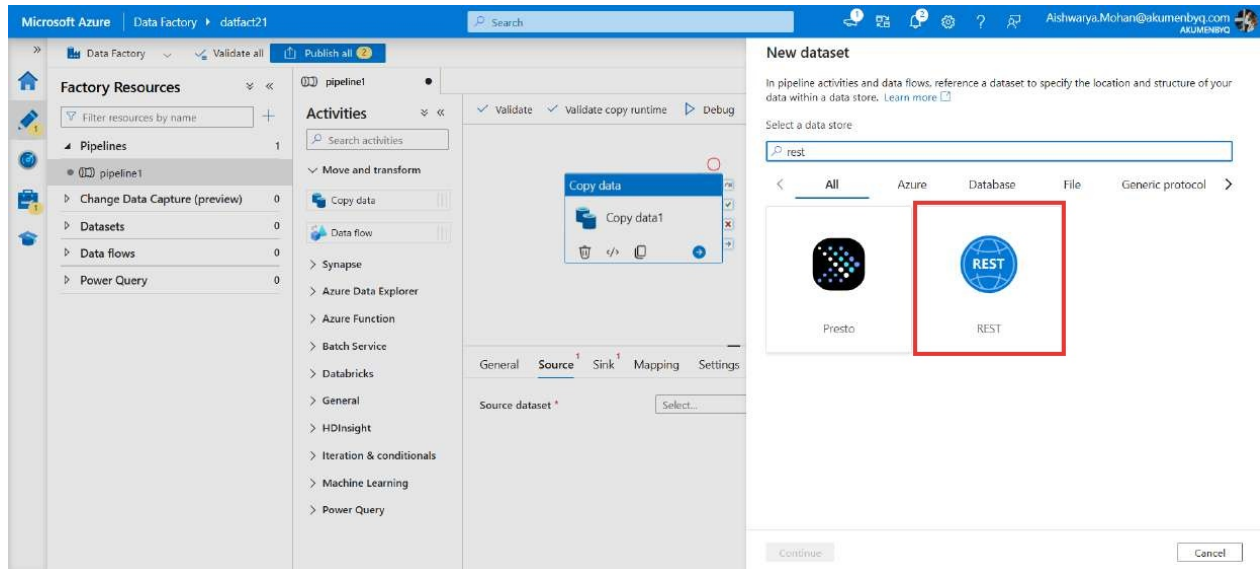
Before: Data Factory had no idea the API or the storage account existed.

After: two linked services are live — LS_REST and LS_ADLS — the plumbing is in place, and it's time to build the actual pipeline.

Part 3b — The Pipeline: Fetching the News

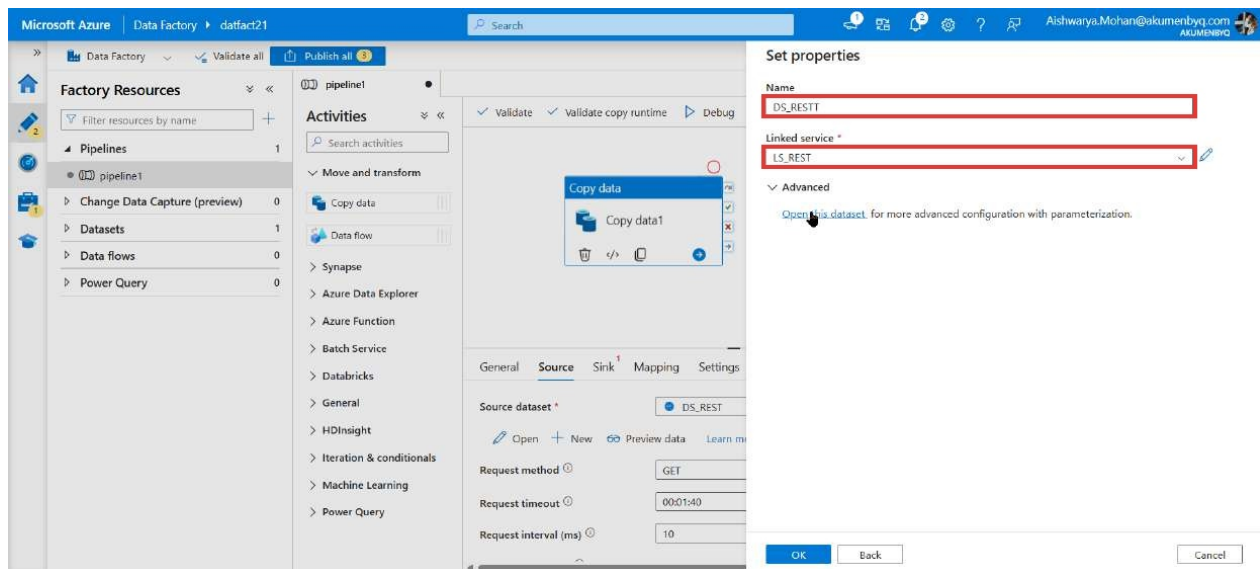
A pipeline in Data Factory is a sequence of activities. This one only needs a single **Copy Data** activity: read from the REST API (source), write to ADLS (sink). Let's wire up the source side first.

Step 14: Create the source dataset



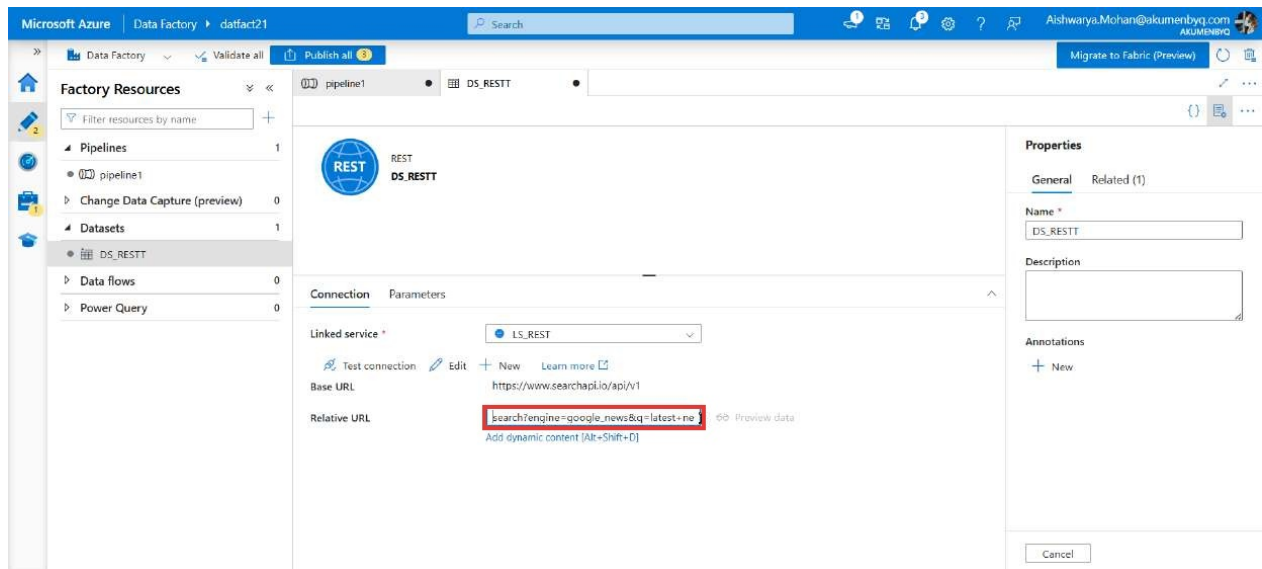
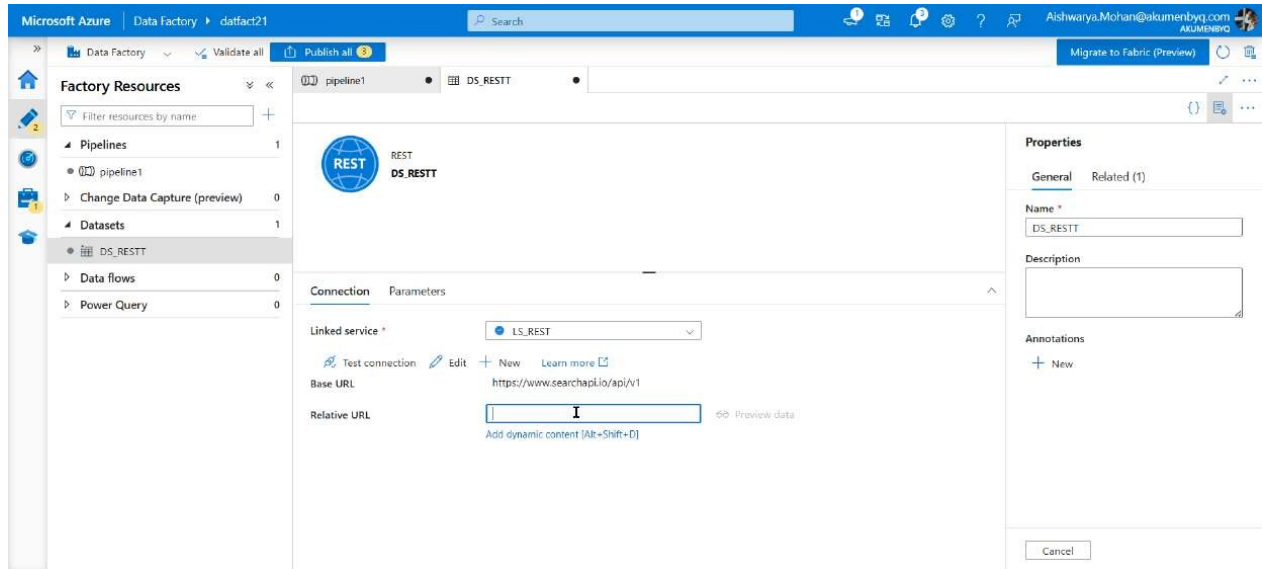
A dataset in Data Factory describes exactly what data to read or write, using one of the linked services as its connection. For the source, I picked the REST type again — this dataset will represent "the SearchAPI request."

Step 15: Name it and link it



Named it DS_RESTT and pointed it at LS_REST, the connection created in Step 11.

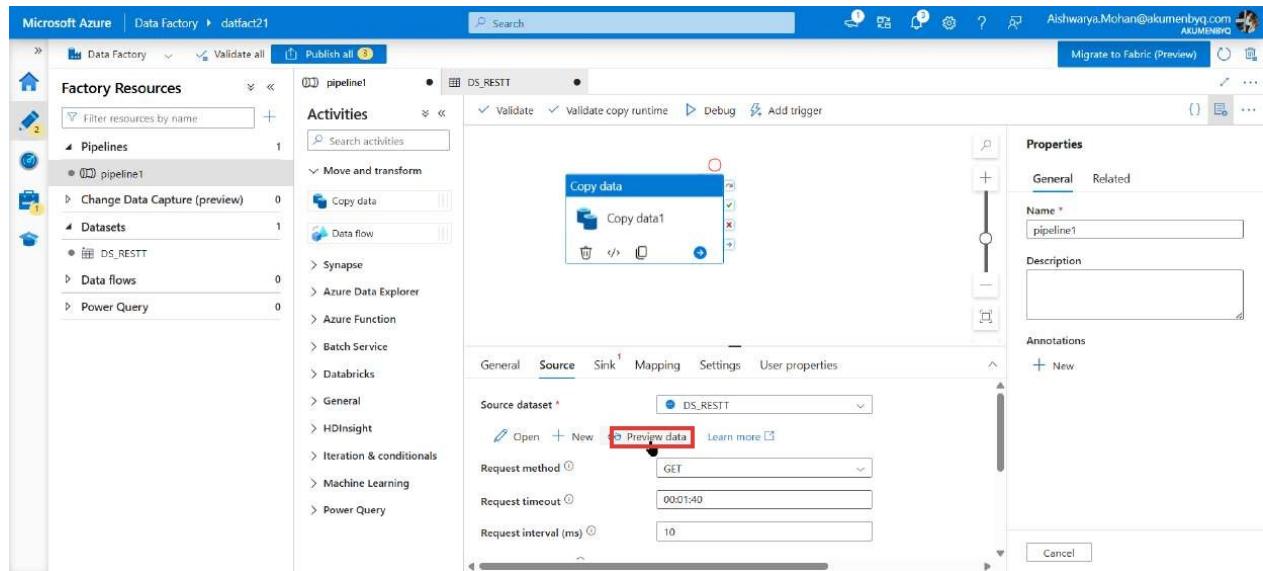
Step 16: Paste in the rest of the URL



Before: the dataset only knows the Base URL (searchapi.io/api/v1) — the Relative URL box is empty.

After: pasting in search?engine=google_news&q=latest+news&gl=in&api_key=... as the Relative URL completes the exact same request URL built back in Step 6. Base URL + Relative URL = the full API call.

Step 17: Preview the data



Before running anything for real, the **Preview data** button on the Copy activity lets you sanity-check the request. Request method is GET, which is all SearchAPI needs.

Step 18: Check it matches the real Google News site

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with 'pipeline1' selected. The main area displays a 'Preview data' window for 'pipeline1', showing a JSON array of news items. The JSON structure includes fields for 'title', 'link', 'source', 'date', 'iso_date', 'snippet', 'is_live', 'favicon', and 'thumbnail'. The right-hand 'Properties' pane shows the 'General' tab with the pipeline name 'pipeline1' and a description field.

```

{
  "title": "Nepal flash floods LIVE updates: Risk of lake 'burst' near border disaster site wanes, 'A",
  "link": "https://www.google.com/goto?url=CAES5gEB6zswFQmJv6wK974bd4-3gF2CaK2k29nIUgrzuv3FjgTpS4",
  "source": "The Hindu",
  "date": "1 hour ago",
  "iso_date": "2026-08-29T07:22:33Z",
  "snippet": "Follow The Hindu for LIVE updates on the Nepal flash floods, which have left hundreds",
  "is_live": true,
  "favicon": "data:image/png;base64,iVBORw0KGgoAAAANSUkEgAAABAAAAQCAVAAAAoLQ9TAAASf8WVEX///wsh3I",
  "thumbnail": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQAAQABAAQ/2wCEAAAGBwgH8gK1BwgKCgkLDRYPDQwM",
  "position": 3,
  "title": "Trump announces 'biggest oil deal in world history' with Venezuela",
  "link": "https://www.google.com/goto?url=CAES5gEB6zswF50CHkv8Xgbv27Zptl15WaY66ccQ5y7MKc44M8vI72Q",
  "source": "Al Jazeera",
  "date": "6 hours ago",
  "iso_date": "2026-08-29T02:36:57Z",
  "snippet": "US president says deal secures majority control of more than 65 billion barrels of pro",
  "favicon": "data:image/png;base64,iVBORw0KGgoAAAANSUkEgAAABAAAAQCAVAAAAoLQ9TAAAV18HVEX6KAD6Jw0",
  "thumbnail": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQAAQABAAQ/2wCEAAAGBwgH8gK1BwgKCgkLDRYPDQwM",
  "position": 4,
  "title": "RSS does not represent us: US-based Hindu group among 61 organisations protesting Moh",
  "link": "https://www.google.com/goto?url=CAES5gEB6zswF5-aRXv3M-6IeXo5gFbc5agnu8NZUsQ7NT26g8NAjX",
  "source": "Hindustan Times",
  "date": "15 hours ago",
  "iso_date": "2026-08-28T17:24:25Z",
  "snippet": "RSS chief Mohan Bhagwat is scheduled to address the Universal Oneness Celebrations ew",
  "favicon": "data:image/png;base64,iVBORw0KGgoAAAANSUkEgAAABAAAAQCAVAAAAoLQ9TAAAN018WVEXu/////B",
  "thumbnail": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQAAQABAAQ/2wCEAAAGBwgH8gK1BwgKCgkLDRYPDQwM"
}

```

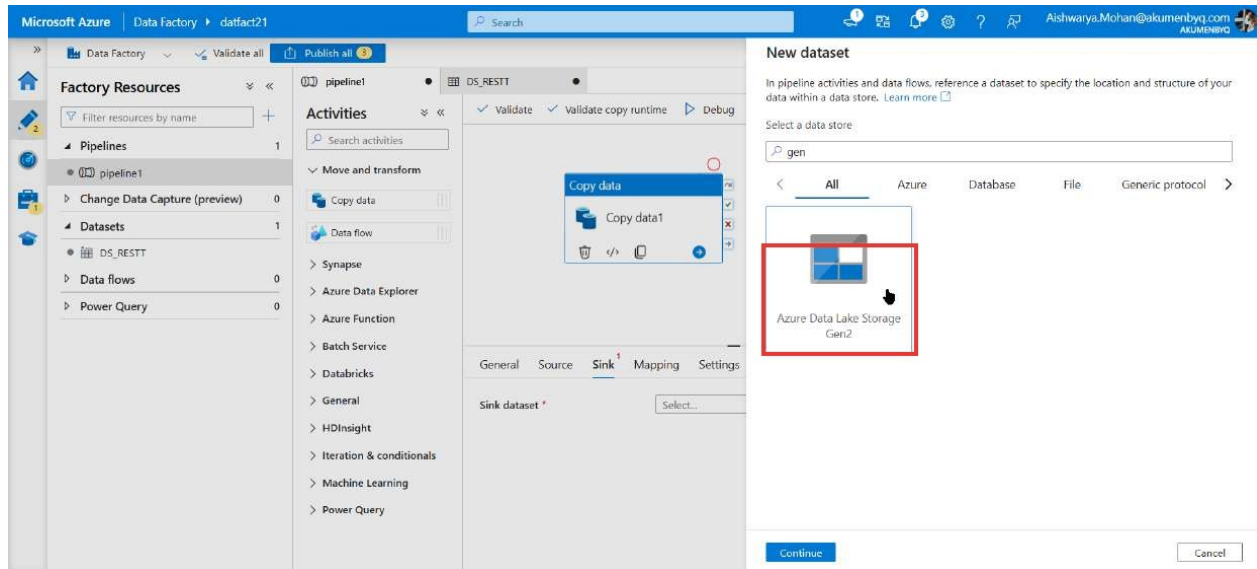
The screenshot shows the Google News website interface. The top navigation bar includes 'Home', 'For you', 'Following', 'News Showcase', and various topic categories like 'India', 'World', 'Local', 'Business', 'Technology', 'Entertainment', 'Sports', 'Science', and 'Health'. The main content area displays a headline titled 'Nepal flood death toll >' with a sub-headline 'Nepal floods latest: Death toll jumps to 616 as search for survivors enters fourth day - follow live'. Below this, there are several news snippets from different sources, including 'The Hindu', 'The Times of India', 'Rediff', and 'News On AIR'. The snippets provide updates on the Nepal flash floods, the reconstruction cost, and the death toll.

The preview returns a clean JSON array — title, link, source, date, and snippet for each headline. Comparing it against Google News itself confirms the API is genuinely pulling live, current headlines and not stale or fake data.

Part 3c — The Pipeline: Saving the Output

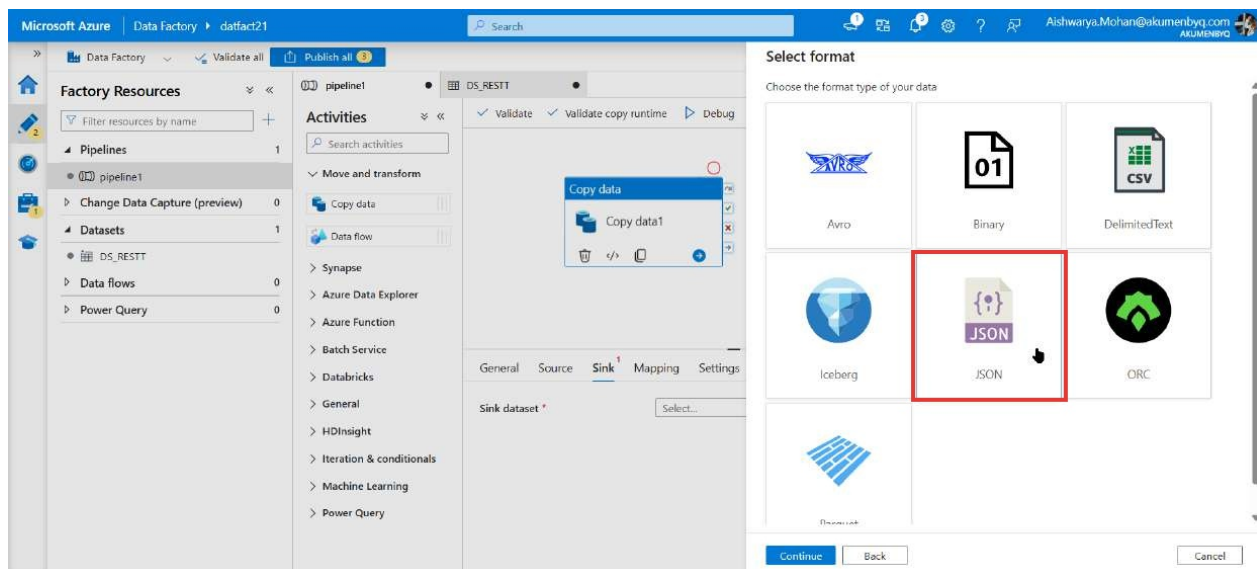
With the source side fetching real data, the sink side needs to decide two things: what format to save it in, and where exactly (which folder, which filename).

Step 19: Create the sink dataset (ADLS Gen2)



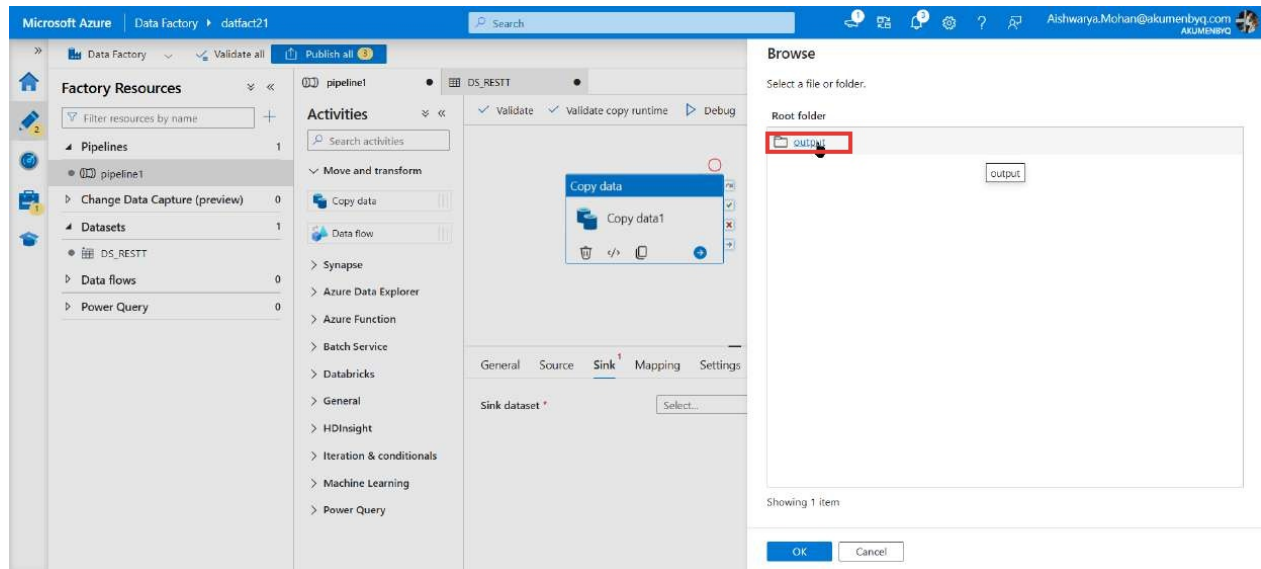
Same idea as the source dataset, but this time pointing at LS_ADLS — the storage account.

Step 20: Choose JSON as the format



Since the API already returns JSON, there's no reason to convert it into anything else — saving it as JSON keeps the data exactly as-is.

Step 21: Point it at the output container

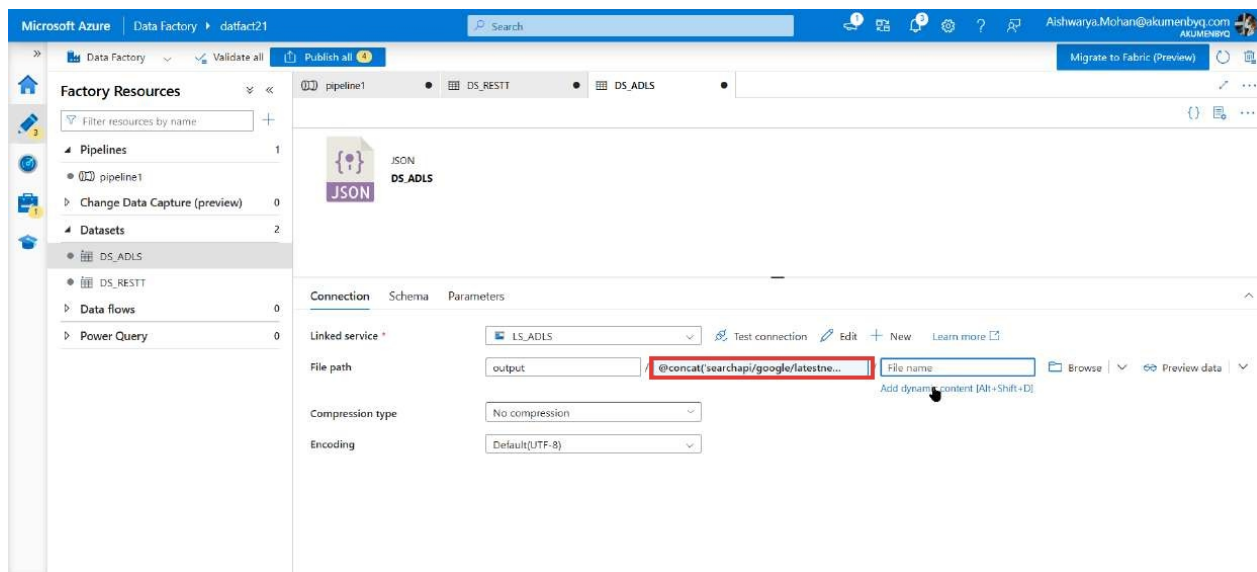


Browsing into the storage account and selecting the output container created back in Step 9 sets the root folder for everything this pipeline saves.

Step 22: Auto-organize files into dated folders

This screenshot shows the Microsoft Azure Data Factory portal interface. On the left, the 'Factory Resources' pane lists 'Pipelines' (1), 'Datasets' (2), and 'Data flows' (0). The 'DS_ADLS' dataset is selected. The main pane displays the configuration for the 'JSON' dataset, with the 'Connection' tab active. The 'Linked service' is set to 'LS_ADLS'. The 'File path' is configured as 'output / Directory / File name', where 'Directory' is highlighted with a blue selection box and a tooltip that reads 'Add dynamic content (Alt+Shift+D)'. The 'Compression type' is set to 'No compression' and the 'Encoding' is 'Default (UTF-8)'. On the right, the 'Properties' pane shows the 'Name' as 'DS_ADLS' and a 'Description' field. A 'Cancel' button is at the bottom right.

This screenshot shows the same Microsoft Azure Data Factory portal interface, but with the 'Pipeline expression builder' dialog open on the right. The dialog has a title bar 'Pipeline expression builder' and a subtitle 'Add dynamic content below using any combination of expressions, functions and system variables:'. The main text area contains the expression: `@concat('searchapi/google/latestnews/',formatDateTime(UTCNow(), 'yyyy/mm/dd'))`, which is highlighted with a red rectangular box. Below the text area is a 'Clear contents' link. At the bottom of the dialog are 'Parameters' and 'Functions' tabs, a search bar, and 'OK' and 'Cancel' buttons. The background configuration of the dataset remains visible but slightly dimmed.

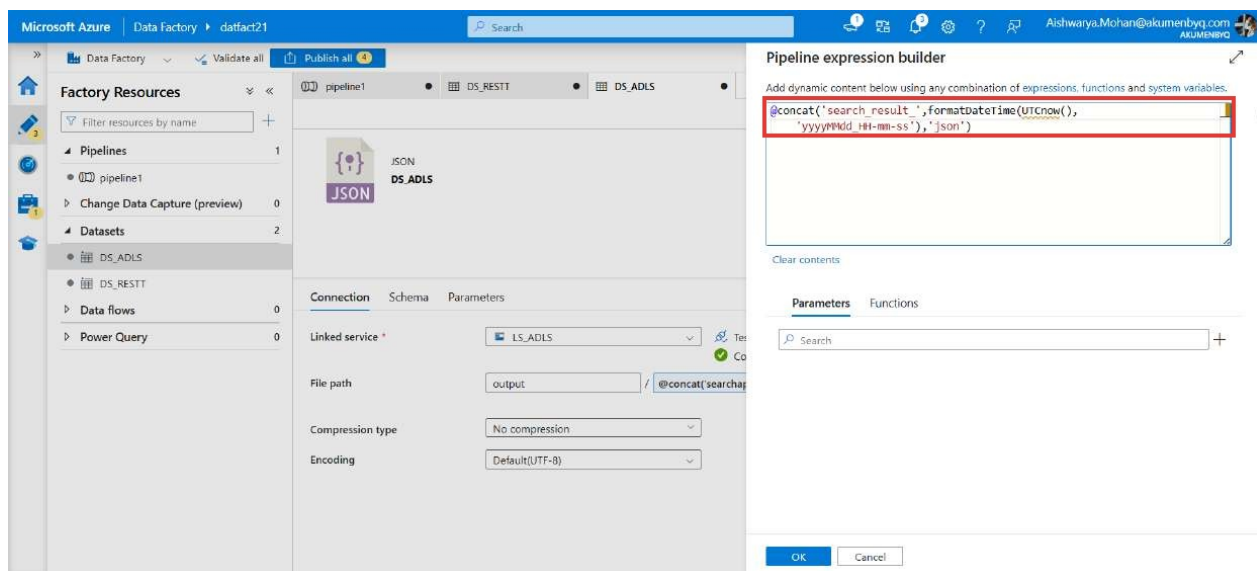


Before: the Directory and File name fields are both empty — the pipeline doesn't know where inside "output" to save things.

After: using a dynamic expression — `@concat('searchapi/google/latestnews/', formatDateTime(utcnow(), 'yyyy/mm/dd'))` — the Directory field builds itself fresh every run, sorting files into folders by year/month/day automatically. No manual renaming, ever.

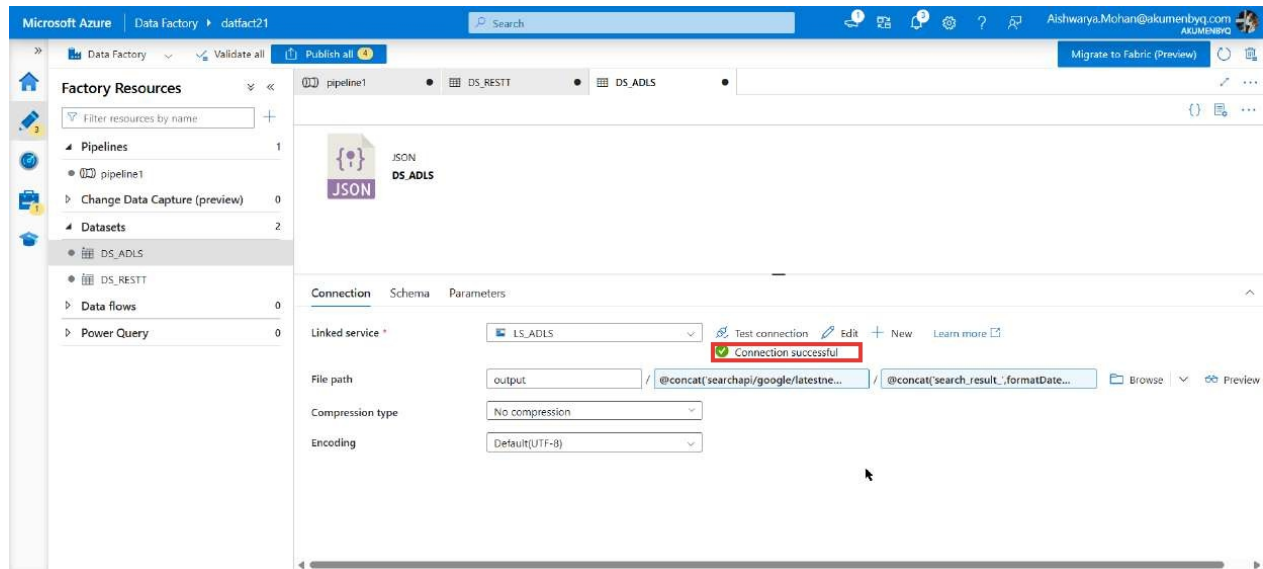
Gotcha #1: date-format tokens are case-sensitive. Lowercase mm actually means *minutes*, not month — I had to go back and capitalize it to MM to get real year/month/day folders.

Step 23: Build a unique filename, and fix another gotcha



Gotcha #2: my first attempt at the filename used colons in the timestamp (HH:MM:SS), but Azure file paths can't contain colons — it flagged the expression as invalid. Swapping to dashes fixed it: `@concat('search_result_', formatDateTime(utcnow(), 'yyyyMMdd_HH-mm-ss'), '.json')`. Small detail, easy to miss the first time.

Step 24: Test the connection

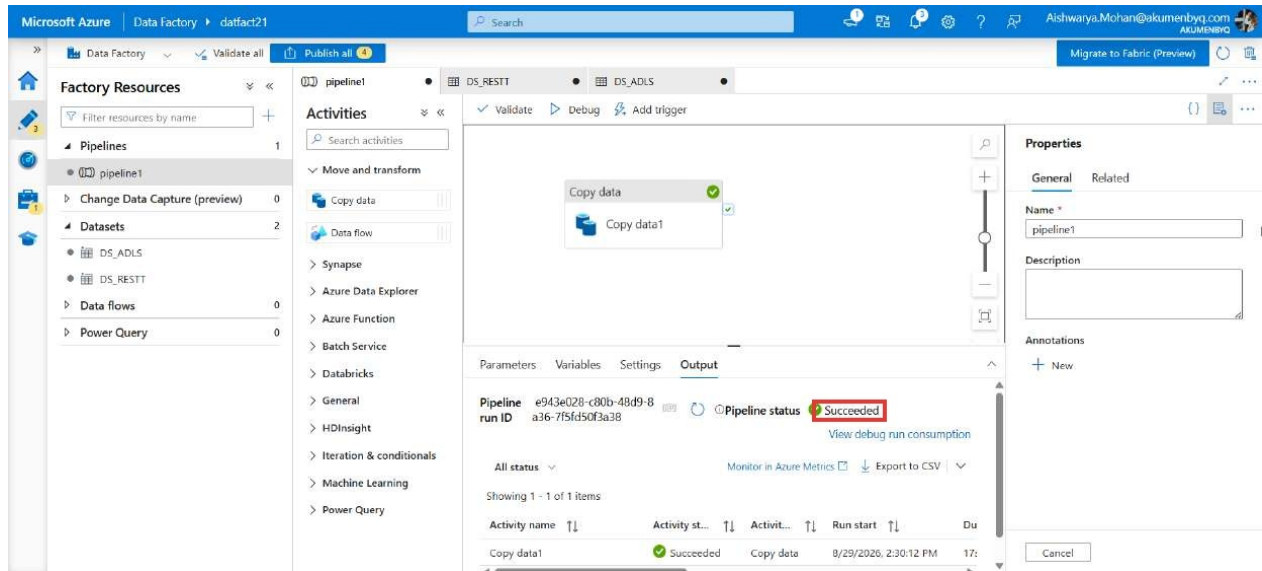


With everything filled in, **Test connection** comes back green — "Connection successful." That's confirmation the sink dataset can actually reach the storage account and write there.

Part 4 — Running It and Checking the Result

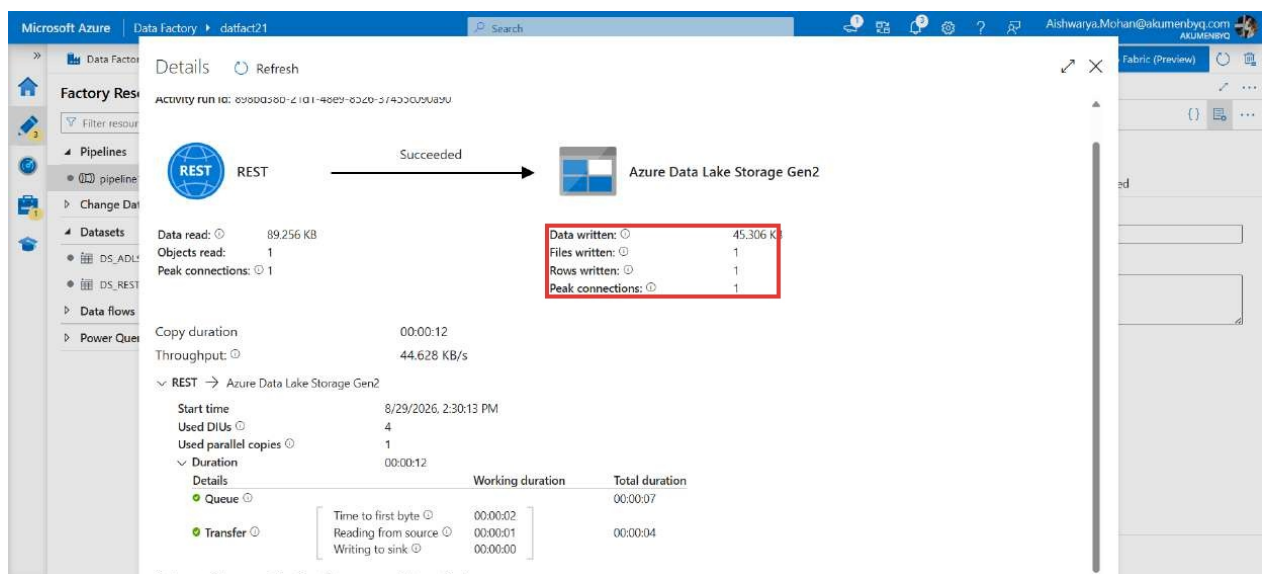
Everything's wired up: source fetches the news, sink saves it. Time to actually run the pipeline and see if it works end to end.

Step 25: Run it



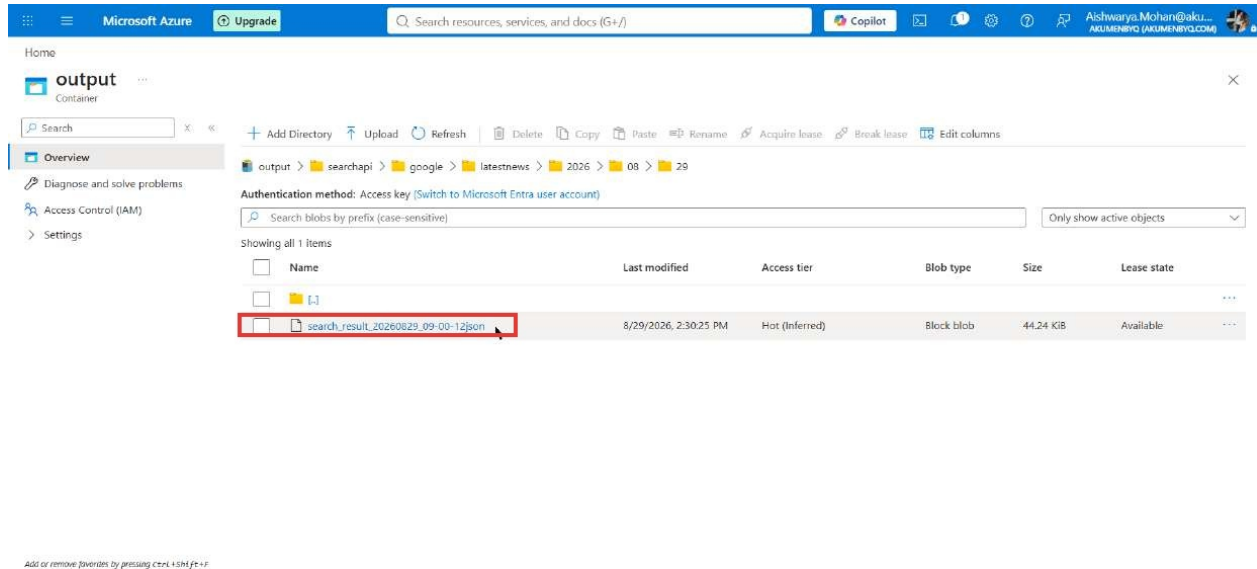
Hitting Debug (or a real trigger, later) kicks off the Copy Data activity. The Output tab shows the pipeline status flip to a green **Succeeded**.

Step 26: Check the copy statistics



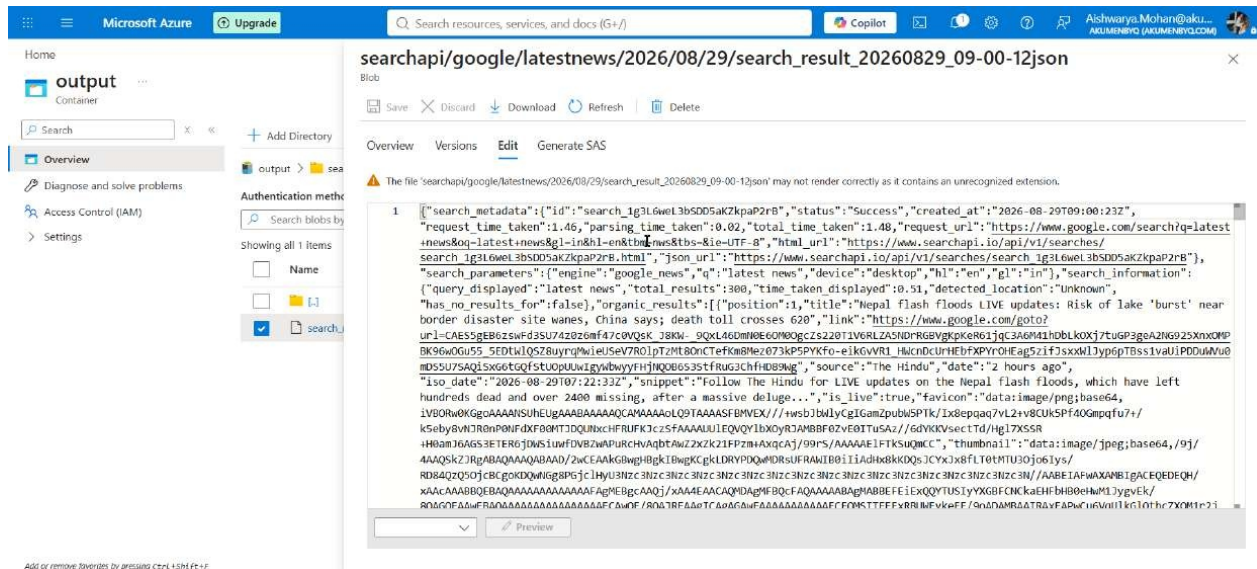
Clicking into the activity's details shows the numbers: about 89 KB read from the API, about 45 KB written to storage, 1 file written, 1 row written, and the whole thing took just 12 seconds. Fast, and exactly what was expected.

Step 27: Find the file in storage



Following the breadcrumb — output > searchapi > google > latestnews > 2026 > 08 > 29 — leads straight to the file, auto-named search_result_20260829_09-00-12.json. That's the dynamic Directory and File name expressions from Steps 22–23 doing exactly what they were built to do.

Step 28: Peek inside the file



Opening the blob directly in Azure confirms it's a full, valid JSON response — search metadata, the parameters used, and the list of news articles with titles, links, sources, dates, and snippets. From here, this file is ready to be picked up by anything downstream: a Databricks notebook, a Power BI report, or another pipeline entirely.

End to end: one API URL, two Azure linked services, one pipeline, and a fresh Google News snapshot saved automatically every time it runs.